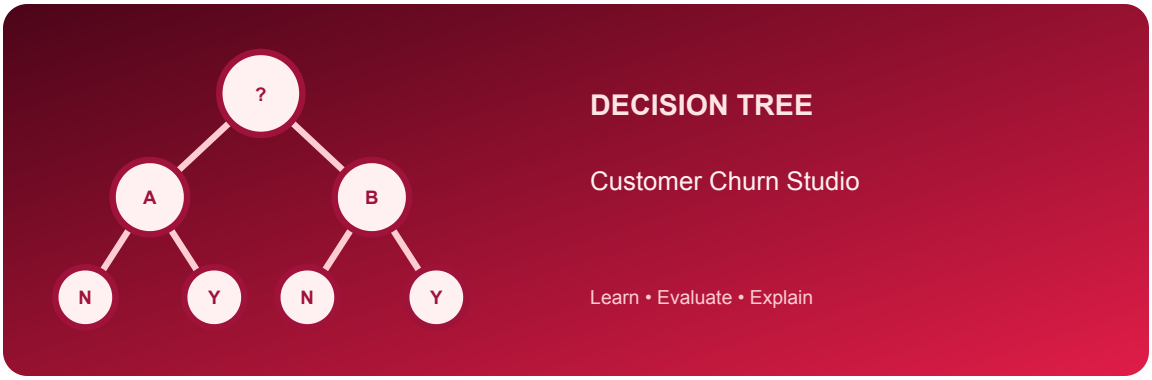
🌳 Telecom Customer Churn



Decision Tree · Customer Experience

A zero-to-deployment learning workbook
Project 1 of 2 in the Decision Tree chapter.

What you will build

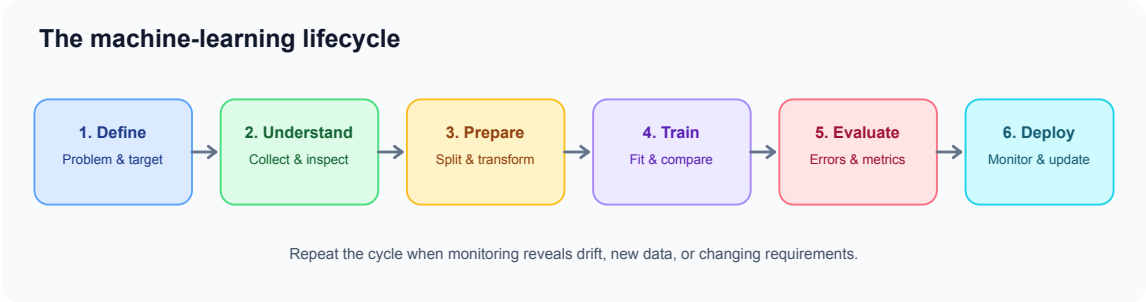
A machine-learning pipeline that predicts the historical churn label in a sample telecommunications dataset. You will learn the theory, inspect data, prevent leakage, build a baseline, train a Decision Tree, evaluate it, track an experiment, save a reusable pipeline, test artifact loading, and later connect it to Streamlit.

Important: this is an educational demonstration, not a validated system for making consequential decisions about customers. Predictive association is not causal explanation, and test metrics do not guarantee performance on future customers.

How to study this notebook

- Read every Markdown cell before running the code that follows it.
- Run cells from top to bottom and inspect every result.
- If a cell fails, fix the error before continuing; do not hide it.
- After successful execution, save the notebook so code and output cells are both stored.
- Export the executed notebook to HTML and PDF using the publishing script and inspect both before sharing.

Machine-learning lifecycle in this project



Phase	What we do	Evidence of completion
1. Problem definition	Define the target, inputs, intended use and risks	Written problem statement
2. Data acquisition	Locate source and review terms/licence	Source and terms documented
3. Data understanding	Inspect schema, quality and class balance	Tables and plots
4. Preparation	Handle missing values and encode categories	Reusable preprocessing pipeline
5. Split	Hold out test rows before fitting transformations	Reproducible stratified split
6. Training	Fit baseline and Decision Tree	Fitted pipeline
7. Evaluation	Analyse metrics, errors and trade-offs	Metrics and confusion matrix
8. Experiment tracking	Record parameters and metrics in MLflow	Recorded run
9. Packaging	Save pipeline and metadata	Artifacts reload successfully
10. Testing	Check prediction shape, schema and loading	Passing tests/checks
11. Deployment	Integrate with Streamlit and publish	Live URL tested separately
12. Maintenance	Watch drift, quality and performance	Monitoring/retraining plan

1. Problem definition: what, why, and when?

What is classification?

Classification predicts a **discrete label**. Here, the label is `Churn`, commonly `Yes` or `No`. The model learns patterns in historical examples and estimates which class best fits a new row.

What is a Decision Tree?

A Decision Tree uses a sequence of feature-based questions. Each internal node applies a rule, each branch represents an outcome of that rule, and each leaf produces a

prediction. Think of a troubleshooting flowchart that asks one question at a time.

Why use a tree?

- It can represent nonlinear patterns and interactions between features.
- It can work with numeric and categorical inputs after appropriate encoding.
- It does not require numeric standardisation in the way distance-based methods often do.
- Individual paths can often be explained to learners and stakeholders.

When is it useful?

Use it as a tabular classification baseline when relationships may be nonlinear, when rule paths are valuable, or when you want to learn the foundations of tree ensembles such as Random Forest and Gradient Boosting.

When should we be cautious?

Single trees can be unstable: small data changes can produce a different tree. Deep trees can memorise noise. Imbalanced labels can make accuracy misleading. Identifiers, target-derived fields, and data available only after the prediction time can cause leakage. Feature importance is not proof of causation.

Our problem statement

Train a reproducible educational classifier to estimate the historical churn label from features available at prediction time. Exclude the customer identifier and any information that directly reveals the answer. We will report mistakes and metric trade-offs rather than claiming that a high score makes the system safe or useful on its own.

2. Mathematical foundations

At each node, the algorithm searches candidate feature splits and evaluates how much they reduce class mixing. The choice of impurity function and tree constraints influence the learned structure.

Gini impurity

For a node t with K classes and class proportions $p_1, \ldots, p_K$:

$$G(t) = 1 - \sum_{k=1}^{K} p_k^2$$

For two classes with proportions p and $1 - p$:

$$G(t) = 1 - p^2 - (1-p)^2 = 2p(1-p)$$

Gini is 0 for a pure node. For a binary node it reaches 0.5 when both classes are equally common.

Entropy

$$H(t) = -\sum_{k=1}^{K} p_k \log_2(p_k)$$

We define $0 \log_2(0) = 0$. Entropy is also zero for a pure node; it rises as class membership becomes more uncertain.

Weighted impurity after splitting

If a parent with N observations is split into children $t_1, \ldots, t_m$, where child j contains N_j rows:

$$I_{after} = \sum_{j=1}^{m} \frac{N_j}{N} I(t_j)$$

Here, I can be Gini or entropy. The weighting matters because a child containing 90% of the rows should count more than a child containing 10%.

Impurity reduction

$$\Delta I = I(parent) - I_{after}$$

The tree prefers a split with a larger impurity reduction, subject to implementation details and configured constraints. Scikit-learn supports Gini and entropy criteria; we will compare performance rather than claim one is always superior.

Prediction at a leaf

The class prediction is typically the most represented weighted class in a leaf. A probability output is based on class proportions in that leaf. Such probabilities are not guaranteed to be calibrated, especially for small leaves or shifted data.

Regularisation: controlling complexity

- `max_depth` : limits the number of levels.
- `min_samples_split` : minimum rows needed to split a node.
- `min_samples_leaf` : minimum rows in each leaf.
- `max_leaf_nodes` : caps the number of terminal leaves.
- `ccp_alpha` : cost-complexity pruning strength; higher values prune more.

These controls trade variance against bias. A large tree can learn complex rules but may overfit; a small tree may miss real patterns. Select hyperparameters using training-only cross-validation or a validation set. Keep the final test set for evaluation after the choices are made.

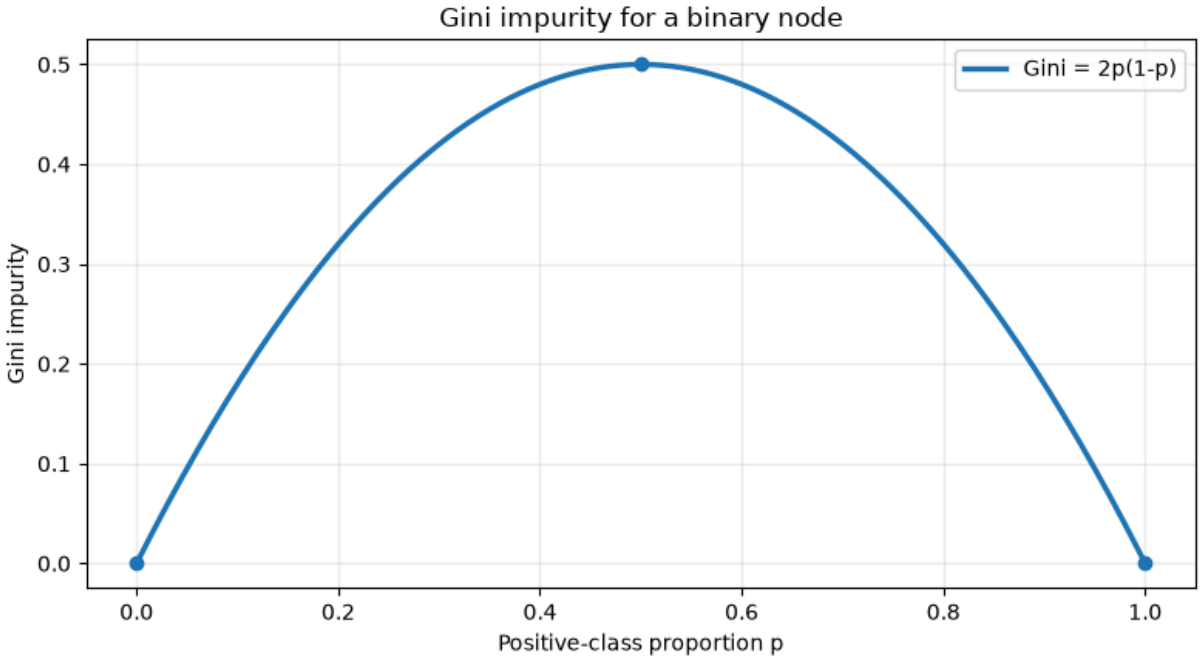
In [1]:
```python
# Work through Gini by hand before asking a library to train a tree.
# Example node: 6 churn records and 4 non-churn records.
n_churn, n_not_churn = 6, 4
p_churn = n_churn / (n_churn + n_not_churn)
p_not_churn = n_not_churn / (n_churn + n_not_churn)
gini = 1 - (p_churn**2 + p_not_churn**2)

print(f"P(churn)     = {p_churn:.2f}")
print(f"P(not churn) = {p_not_churn:.2f}")
print(f"Gini impurity = {gini:.3f}")
assert abs(gini - 0.48) < 1e-12
```

```
P(churn)     = 0.60
P(not churn) = 0.40
Gini impurity = 0.480
```

In [2]:
```python
# Visualise binary Gini impurity as the positive-class fraction changes.
import numpy as np
import matplotlib.pyplot as plt

p = np.linspace(0, 1, 201)
gini_curve = 2 * p * (1 - p)
fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot(p, gini_curve, linewidth=2.5, label="Gini = 2p(1-p)")
ax.scatter([0, 0.5, 1], [0, 0.5, 0], zorder=3)
ax.set(title="Gini impurity for a binary node", xlabel="Positive-class propo
ax.grid(alpha=0.25)
ax.legend()
plt.tight_layout()
plt.show()
```



3. Dataset source, data rights, and reproducibility

Candidate dataset: IBM's sample Telco Customer Churn CSV.

- Context: IBM telecom churn tutorial
- Candidate CSV: Telco-Customer-Churn.csv

The target is commonly called `Churn`, with values such as `Yes` and `No`. We must inspect the actual downloaded file to verify columns and row counts.

Data rights warning: a code repository's licence does not automatically grant rights to redistribute a separate dataset. Check the actual dataset terms before publishing raw data. This notebook downloads a local copy for learning but does not automatically commit the CSV.

If the source URL fails or its terms cannot be established, stop and verify the source rather than silently replacing it.

In [3]:
```
from pathlib import Path
from urllib.request import urlopen

DATA_DIR = Path("data")
DATA_DIR.mkdir(parents=True, exist_ok=True)
DATA_PATH = DATA_DIR / "Telco-Customer-Churn.csv"
DATA_URL = "https://raw.githubusercontent.com/IBM/telco-customer-churn-on-ic

if DATA_PATH.exists():
    print(f"Using existing file: {DATA_PATH}")
else:
    print("Downloading the documented candidate dataset...")
    with urlopen(DATA_URL, timeout=30) as response:
```

```
        DATA_PATH.write_bytes(response.read())

print(f"File exists: {DATA_PATH.exists()}")
print(f"File size: {DATA_PATH.stat().st_size:,} bytes")
```

```
Using existing file: data/Telco-Customer-Churn.csv
File exists: True
File size: 970,457 bytes
```

4. Inspect the data before modelling

First check the dataset shape, types, missing values, duplicates, numerical summaries and target balance. Data inspection catches wrong delimiters, unexpected labels, blank fields, and type problems before they propagate into training.

`TotalCharges` may contain blank strings and therefore be read as text. We convert it to numeric later, letting unparseable values become missing so they can be handled by the training-only imputer. `customerID` is an identifier and should not be used to predict churn.

In [4]:
```
import pandas as pd

df = pd.read_csv(DATA_PATH)
print(f"Rows: {df.shape[0]:,} | Columns: {df.shape[1]:,}")
display(df.head())
display(df.dtypes.astype(str).rename("dtype").to_frame())
display(df.isna().sum().sort_values(ascending=False).rename("missing_count")
print(f"Duplicate rows: {df.duplicated().sum():,}")

if "Churn" not in df.columns:
    raise ValueError("The expected 'Churn' target is missing; inspect the dc

display(df["Churn"].value_counts(dropna=False).rename("count").to_frame())
ax = df["Churn"].value_counts(normalize=True).plot(kind="bar", title="Churn
ax.set_xlabel("Historical churn label")
ax.set_ylabel("Proportion of rows")
plt.tight_layout()
plt.show()
```

```
Rows: 7,043 | Columns: 21
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Mul
0	7590-VHVEG	Female	0	Yes	No	1	No	
1	5575-GNVDE	Male	0	No	No	34	Yes	
2	3668-QPYBK	Male	0	No	No	2	Yes	
3	7795-CFOCW	Male	0	No	No	45	No	
4	9237-HQITU	Female	0	No	No	2	Yes	

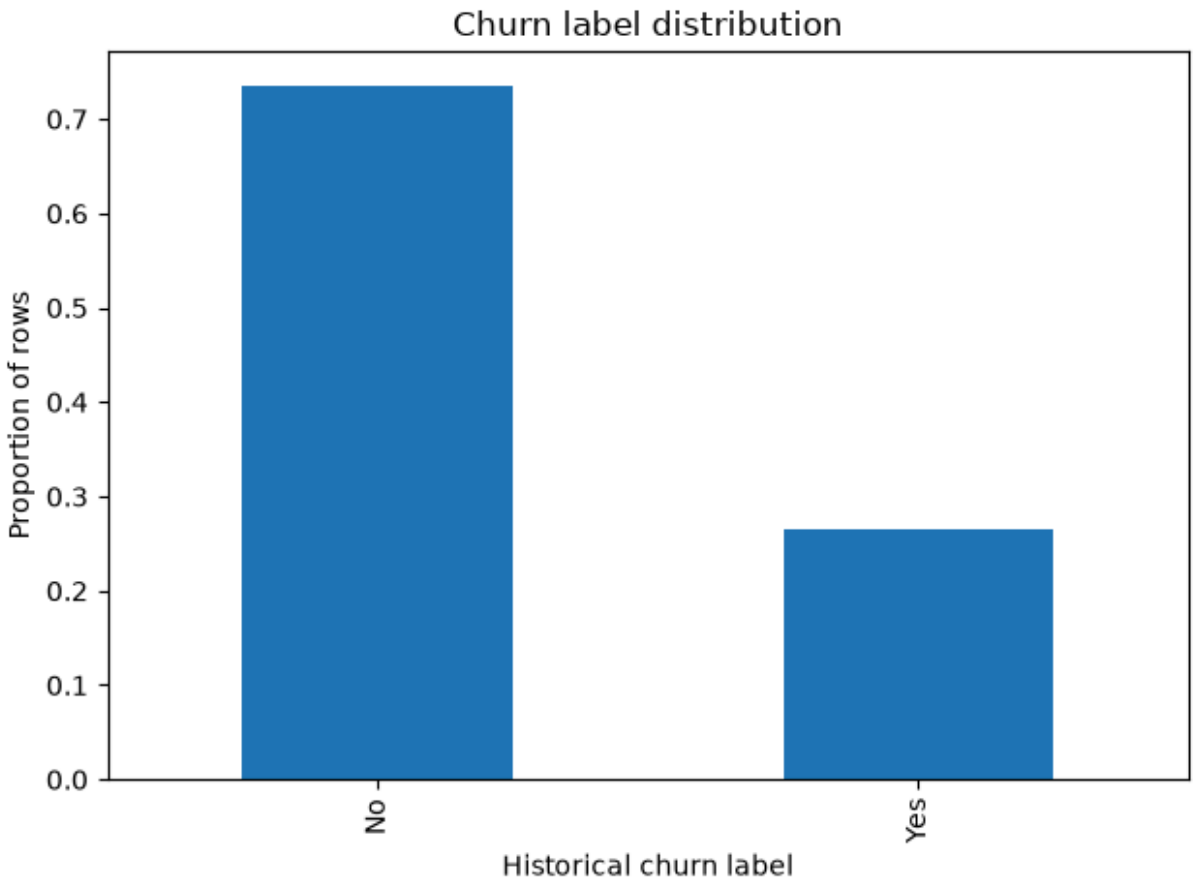
5 rows × 21 columns

	dtype
customerID	object
gender	object
SeniorCitizen	int64
Partner	object
Dependents	object
tenure	int64
PhoneService	object
MultipleLines	object
InternetService	object
OnlineSecurity	object
OnlineBackup	object
DeviceProtection	object
TechSupport	object
StreamingTV	object
StreamingMovies	object
Contract	object
PaperlessBilling	object
PaymentMethod	object
MonthlyCharges	float64
TotalCharges	object
Churn	object

	missing_count
customerID	0
DeviceProtection	0
TotalCharges	0
MonthlyCharges	0
PaymentMethod	0
PaperlessBilling	0
Contract	0
StreamingMovies	0
StreamingTV	0
TechSupport	0
OnlineBackup	0
gender	0
OnlineSecurity	0
InternetService	0
MultipleLines	0
PhoneService	0
tenure	0
Dependents	0
Partner	0
SeniorCitizen	0
Churn	0

```
Duplicate rows: 0
```

Churn	count
No	5174
Yes	1869



5. Build a clean modelling table and split the data

We remove exact duplicate rows, separate the target, and exclude `customerID`. The target needs to contain the two expected labels. Then we create a stratified hold-out test set so the proportion of churn/non-churn examples is approximately preserved.

All learned preprocessing will be fitted inside a pipeline after the split. That means imputation statistics and category encodings are learned only from training rows—not from the test set. This is one of the key protections against data leakage.

We use an 80/20 split and a fixed random seed so the exercise can be reproduced. The final test set must not be used repeatedly to choose hyperparameters.

In [5]:
```
from sklearn.model_selection import train_test_split

data = df.copy()
if "TotalCharges" in data.columns:
    data["TotalCharges"] = pd.to_numeric(data["TotalCharges"], errors="coerc
data = data.drop_duplicates().copy()

y = data["Churn"].astype(str).str.strip()
X = data.drop(columns=["Churn", "customerID"], errors="ignore")
labels = sorted(y.dropna().unique().tolist())
print("Observed target labels:", labels)
```

```
if set(labels) != {"No", "Yes"}:
    raise ValueError(f"Unexpected target labels: {labels}. Review the datase

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
print(f"Training records: {len(X_train):,}")
print(f"Test records:     {len(X_test):,}")
print(f"Train churn rate: {(y_train == 'Yes').mean():.3f}")
print(f"Test churn rate:  {(y_test == 'Yes').mean():.3f}")
```

```
Observed target labels: ['No', 'Yes']
Training records: 5,634
Test records:     1,409
Train churn rate: 0.265
Test churn rate:  0.265
```

6. Preprocessing pipeline and baseline

A baseline answers the important question: does the Decision Tree add value beyond a trivial strategy?

We will use `DummyClassifier(strategy="most_frequent")` as a simple reference. We also prepare a `ColumnTransformer`:

- numeric columns: median imputation;
- categorical columns: most-frequent imputation, then one-hot encoding;
- unknown categories at inference: ignored rather than raising an exception.

Decision Trees do not generally need feature standardisation, so we avoid scaling unless a later experiment justifies it. Keeping transformations inside `Pipeline` means the same operations are applied during training and prediction.

In [6]:
```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f

numeric_features = X_train.select_dtypes(include=["number"]).columns.tolist(
categorical_features = X_train.select_dtypes(exclude=["number"]).columns.tol

numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
])
categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])
preprocessor = ColumnTransformer(transformers=[
    ("numeric", numeric_pipeline, numeric_features),
```

```
    ("categorical", categorical_pipeline, categorical_features),
])

baseline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", DummyClassifier(strategy="most_frequent")),
])
baseline.fit(X_train, y_train)
baseline_pred = baseline.predict(X_test)
print(f"Baseline accuracy:       {accuracy_score(y_test, baseline_pred):.3f}
print(f"Baseline churn precision: {precision_score(y_test, baseline_pred, po
print(f"Baseline churn recall:    {recall_score(y_test, baseline_pred, pos_l
print(f"Baseline churn F1:        {f1_score(y_test, baseline_pred, pos_label
```

```
Baseline accuracy:       0.735
Baseline churn precision: 0.000
Baseline churn recall:    0.000
Baseline churn F1:        0.000
```

7. Train a controlled Decision Tree

We start with a regularised tree rather than allowing unlimited depth. `max_depth=5` limits levels; `min_samples_leaf=20` prevents leaves based on very few observations; `class_weight="balanced"` gives more weight to the minority class during fitting. These are starting values for teaching, not known-optimal settings.

A later step should compare tree settings using cross-validation on the training portion only. Class weighting can improve recall but may also produce more false positives; only held-out evaluation can describe its observed trade-off.

In [7]:
```
from sklearn.tree import DecisionTreeClassifier

tree_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", DecisionTreeClassifier(
        criterion="gini",
        max_depth=5,
        min_samples_leaf=20,
        class_weight="balanced",
        random_state=42,
    )),
])
tree_model.fit(X_train, y_train)
tree_pred = tree_model.predict(X_test)

print(f"Accuracy:  {accuracy_score(y_test, tree_pred):.3f}")
print(f"Precision: {precision_score(y_test, tree_pred, pos_label='Yes', zero
print(f"Recall:    {recall_score(y_test, tree_pred, pos_label='Yes', zero_di
print(f"F1-score:  {f1_score(y_test, tree_pred, pos_label='Yes', zero_divisi
```

```
Accuracy:  0.756
Precision: 0.528
Recall:    0.759
F1-score:  0.623
```

8. Evaluation: choose metrics that match the problem

- **Accuracy:** fraction of all predictions that are correct. It may look high when the majority class dominates.
- **Precision:** of customers predicted to churn, the fraction who actually churned in the test data.
- **Recall:** of customers who actually churned in the test data, the fraction detected.
- **F1:** harmonic mean of precision and recall.
- **ROC-AUC:** evaluates ranking across classification thresholds.
- **Average precision:** summarises precision-recall behaviour and can be informative for an uncommon positive class.

The `Yes` class is treated as positive here. A false positive is a non-churn example predicted as churn; a false negative is a churn example predicted as non-churn. Which error matters more depends on the operational context. We are learning to report that trade-off, not to choose a threshold without stakeholder input.

In [8]:
```
from sklearn.metrics import (
    classification_report,
    ConfusionMatrixDisplay,
    roc_auc_score,
    average_precision_score,
)

print(classification_report(y_test, tree_pred, zero_division=0))
tree_classes = list(tree_model.classes_)
positive_index = tree_classes.index("Yes")
churn_scores = tree_model.predict_proba(X_test)[:, positive_index]
y_test_binary = (y_test == "Yes").astype(int)
print(f"ROC-AUC: {roc_auc_score(y_test_binary, churn_scores):.3f}")
print(f"Average precision: {average_precision_score(y_test_binary, churn_scc

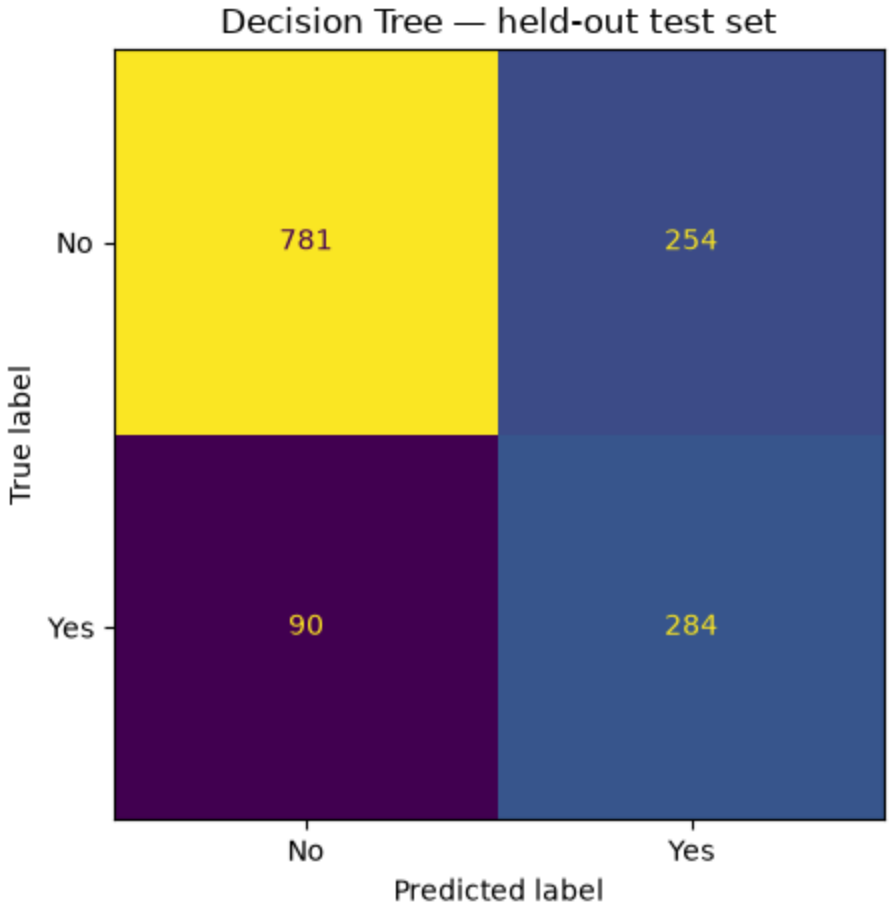
ConfusionMatrixDisplay.from_predictions(
    y_test, tree_pred, labels=["No", "Yes"], colorbar=False
)
plt.title("Decision Tree — held-out test set")
plt.tight_layout()
plt.show()
```

```
              precision    recall  f1-score   support

          No       0.90      0.75      0.82      1035
         Yes       0.53      0.76      0.62       374

    accuracy                           0.76      1409
   macro avg       0.71      0.76      0.72      1409
weighted avg       0.80      0.76      0.77      1409

ROC-AUC: 0.832
Average precision: 0.618
```



9. Visualise the fitted tree and feature importance

A plotted tree makes learned splits visible. Large trees may be unreadable, which is another reason to constrain depth. Impurity-based feature importance reports how much a feature contributed to impurity reduction across the fitted tree; it can be biased by feature types and is not causal evidence.

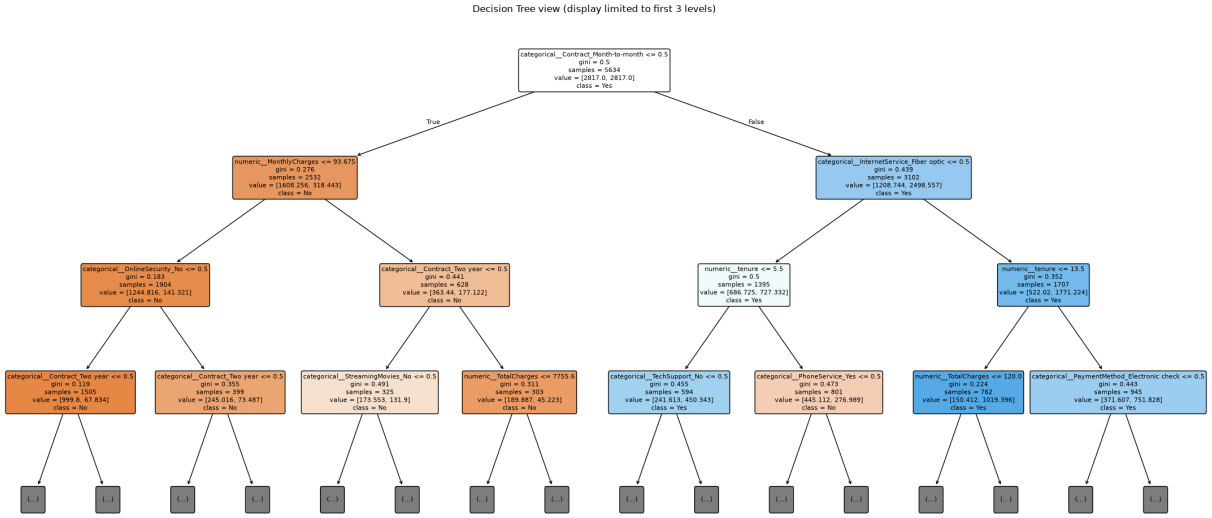
Treat both visuals as model-inspection aids, not proof that a feature causes churn.

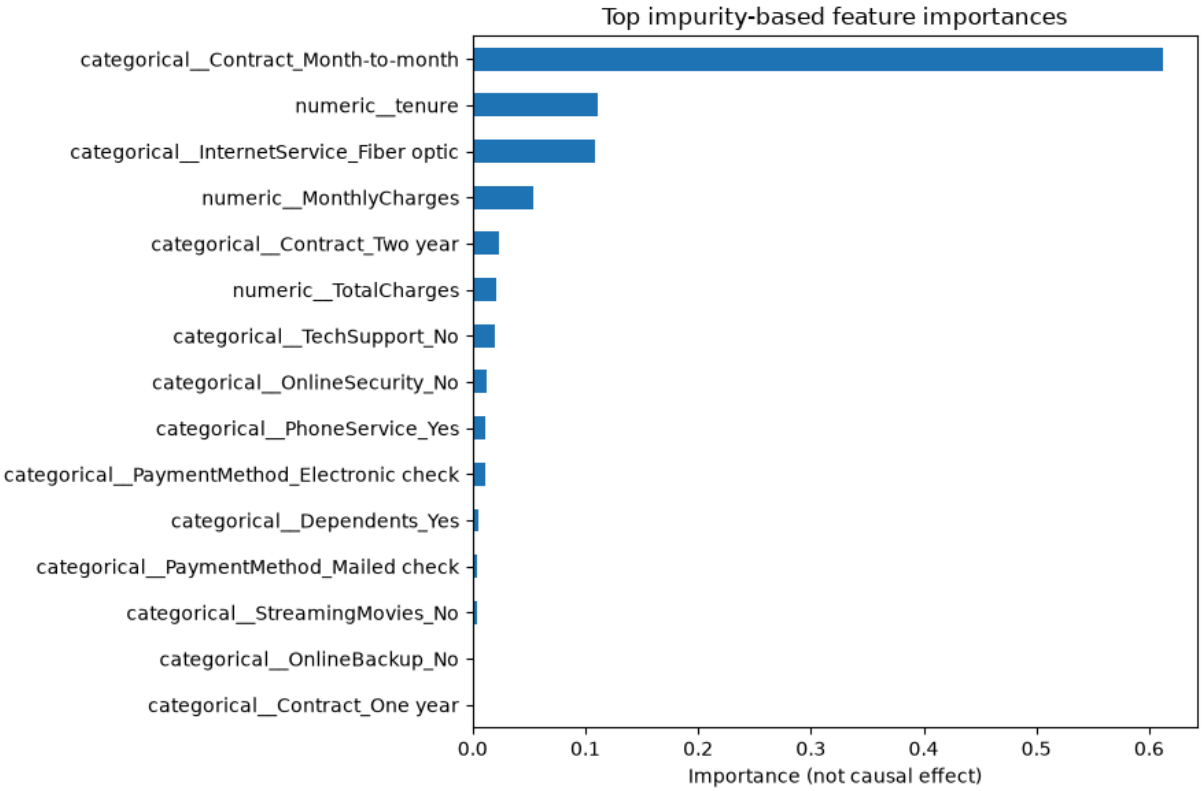
In [9]:
```
from sklearn.tree import plot_tree

fitted_tree = tree_model.named_steps["classifier"]
feature_names = tree_model.named_steps["preprocessor"].get_feature_names_out
```

```python
fig, ax = plt.subplots(figsize=(22, 10))
plot_tree(
    fitted_tree,
    feature_names=feature_names,
    class_names=[str(c) for c in fitted_tree.classes_],
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=8,
    ax=ax,
)
ax.set_title("Decision Tree view (display limited to first 3 levels)")
plt.tight_layout()
plt.show()

importance = pd.Series(
    fitted_tree.feature_importances_, index=feature_names
).sort_values(ascending=False).head(15)
ax = importance.sort_values().plot(
    kind="barh", figsize=(9, 6), title="Top impurity-based feature importanc
)
ax.set_xlabel("Importance (not causal effect)")
plt.tight_layout()
plt.show()
```





10. Experiment tracking with MLflow

MLflow records parameters and metrics so runs can be compared and revisited. The following logs the current controlled tree. Later we can add cross-validation, log additional metrics, and compare alternative criteria and depth settings.

From a project terminal, start the local experiment UI with `mlflow ui` and open the local URL it prints (typically `http://127.0.0.1:5000`). Keep its local tracking files out of Git unless you deliberately configure a shared tracking server.

In [10]:
```
import mlflow
import mlflow.sklearn

mlflow.set_experiment("decision-tree-telecom-churn")
with mlflow.start_run(run_name="depth-5-leaf-20-balanced"):
    mlflow.log_params({
        "criterion": "gini",
        "max_depth": 5,
        "min_samples_leaf": 20,
        "class_weight": "balanced",
        "test_size": 0.20,
        "random_state": 42,
    })
    mlflow.log_metrics({
        "accuracy": accuracy_score(y_test, tree_pred),
        "precision_churn": precision_score(y_test, tree_pred, pos_label="Yes
        "recall_churn": recall_score(y_test, tree_pred, pos_label="Yes", zer
        "f1_churn": f1_score(y_test, tree_pred, pos_label="Yes", zero_divisi
```

```
        "roc_auc": roc_auc_score(y_test_binary, churn_scores),
        "average_precision": average_precision_score(y_test_binary, churn_sc
    })
    # This model is trained locally above. MLflow 3 uses skops by default,
    # so explicitly trust only the audited types required by this pipeline.
    mlflow.sklearn.log_model(
        tree_model,
        name="model",
        skops_trusted_types=[
            "numpy.dtype",
            "sklearn.compose._column_transformer._RemainderColsList",
            "sklearn.tree._tree.Tree",
        ],
    )
    print("MLflow run recorded:", mlflow.active_run().info.run_id)
```

```
2026/10/13 11:22:58 INFO mlflow.agent.hint: Load the `instrumenting-with-mlf
low-tracing` skill at /Users/abhishekpanda/Documents/machine-learning-algori
thms/.venv/lib/python3.12/site-packages/mlflow/assistant/skills/instrumentin
g-with-mlflow-tracing/SKILL.md before writing any tracing code; it ships wit
h this MLflow install. Set MLFLOW_DISABLE_AGENT_HINT=1 to silence this.
MLflow run recorded: 7d86fab177c74c97937765431d119708
```

11. Save a reusable pipeline and metadata

We save the **whole pipeline**: preprocessing plus classifier. Saving only the tree would make it easy for the Streamlit app to apply different cleaning or encoding from training. The metadata records the schema and library versions so we can diagnose incompatibility.

Serialized model files can execute code when loaded. Only load artifacts from sources you trust. Keep the runtime/library versions compatible and never put secrets in metadata.

In [11]:
```
import json
import joblib
import sklearn
import sys

ARTIFACT_DIR = Path("artifacts")
ARTIFACT_DIR.mkdir(parents=True, exist_ok=True)
MODEL_PATH = ARTIFACT_DIR / "decision_tree_churn_pipeline.joblib"
METADATA_PATH = ARTIFACT_DIR / "model_metadata.json"
joblib.dump(tree_model, MODEL_PATH)

metadata = {
    "algorithm": "DecisionTreeClassifier",
    "project": "telecom-customer-churn",
    "target_column": "Churn",
    "positive_class": "Yes",
    "input_features": X.columns.tolist(),
    "classes": [str(value) for value in tree_model.classes_],
    "sklearn_version": sklearn.__version__,
```

```
    "python_version": sys.version.split()[0],
    "dataset_source": DATA_URL,
    "educational_only": True,
    "limitations": [
        "Historical sample data does not guarantee future churn predictions.
        "Feature importance is not causal evidence.",
        "Any real use requires governance, privacy review and subgroup evalu
    ],
}
METADATA_PATH.write_text(json.dumps(metadata, indent=2), encoding="utf-8")
print("Saved pipeline:", MODEL_PATH.resolve())
print("Saved metadata:", METADATA_PATH.resolve())
```

```
Saved pipeline: /Users/abhishekpanda/Documents/machine-learning-algorithms/0
2-decision-tree/01-telecom-customer-churn/artifacts/decision_tree_churn_pipe
line.joblib
Saved metadata: /Users/abhishekpanda/Documents/machine-learning-algorithms/0
2-decision-tree/01-telecom-customer-churn/artifacts/model_metadata.json
```

12. Artifact loading smoke test

Before app development, reload the saved pipeline and predict using raw feature rows. This catches missing artifact files and basic schema mismatches. The tests folder will later contain repeatable versions of this check.

In [12]:
```
loaded_model = joblib.load(MODEL_PATH)
loaded_metadata = json.loads(METADATA_PATH.read_text(encoding="utf-8"))
sample = X_test.head(5)
sample_predictions = loaded_model.predict(sample)
assert len(sample_predictions) == len(sample)
assert set(sample_predictions).issubset(set(loaded_metadata["classes"]))
print("PASS: pipeline and metadata reload successfully.")
display(pd.DataFrame({
    "actual_label": y_test.loc[sample.index],
    "predicted_label": sample_predictions,
}, index=sample.index))
```

```
PASS: pipeline and metadata reload successfully.
```

	actual_label	predicted_label
437	No	No
2280	No	Yes
2235	No	No
4460	No	Yes
3761	No	No

13. What is next: testing, Streamlit and public deployment

This notebook is a training and learning artifact; deployment is a separate checkpoint. Before creating the polished Streamlit UI, verify:

1. Restart the kernel and run every cell from top to bottom.
2. Review metrics and model errors; do not focus on accuracy alone.
3. Confirm both saved artifacts exist and reload correctly.
4. Ensure UI inputs match the recorded raw feature schema and types.
5. Add tests for preprocessing, unseen categories, valid predictions and missing/corrupt artifacts.
6. Confirm the dataset terms, dependencies and app path are documented.
7. Run the app locally, then deploy from the public GitHub repository and test the actual live URL.

The future Streamlit interface will use a consistent polished layout: clear project header, sidebar navigation, labelled inputs, prediction result/probabilities, evaluation insights, dataset source and limitations. It will load the saved pipeline instead of retraining on every page refresh.

14. Generate colourful HTML and PDF with outputs

After the notebook runs successfully, save it in VS Code with its outputs. From the repository root:

```
python tools/export_notebook.py "02-decision-tree/01-telecom-customer-churn/decision_tree_churn.ipynb"
```

The script executes a temporary copy and creates HTML/PDF beside the notebook in the project root. Inspect the files for syntax colouring, formulas, plots, missing images and clipped code. Only then commit the source notebook and reviewed exports for public download.